

Rank 02 — Exam Prep

Compile: `cc file.c -o file` | Test: `./file arg1 arg2` For library functions (`ft_strlen`, `ft_swap`...): compile with a `main.c` or use the provided tester.

Level 0

aff_a

- **File:** `aff_a.c` | **Allowed:** write

Subject

Write a program that takes a string, and displays the first 'a' character it encounters in it, followed by a newline. If there are no 'a' characters in the string, the program just writes a newline. If the number of parameters is not 1, the program displays a newline.

Example:

```
$> ./aff_a "abc" | cat -e
a$
$> ./aff_a "dub0 a POIL" | cat -e
a$
$> ./aff_a "zz sent le poney" | cat -e
$
$> ./aff_a | cat -e
$
```

- **Approach:** Require exactly 1 argument. Scan the string for the first 'a', print it and stop. Always end with `\n`.

```
#include <unistd.h>
int main(int ac, char **av)
{
    int i = 0;
    if (ac == 2) // exactly one argument
        while (av[1][i])
            if (av[1][i] == 'a') { write(1, "a", 1); break; } // print first 'a' and
            ↪ stop
        else i++;
    write(1, "\n", 1); // always print newline
}
```

aff_first_param

- **File:** `aff_first_param.c` | **Allowed:** write

Subject

Write a program that takes strings as arguments, and displays its first argument followed by a newline.

If the number of arguments is less than 1, the program displays a newline.

Examples:

```
$> ./aff_first_param vincent mit "l'ane" dans un pre et "s'en" vint | cat -e
vincent$
$> ./aff_first_param "j'aime le savon"
j'aime le savon
```

```
$> ./aff_first_param
$
```

- **Approach:** Print argv[1] char by char if at least one argument exists, then \n.

```
#include <unistd.h>
int main(int ac, char **av)
{
    int i = 0;
    if (ac > 1) // at least one argument
        while (av[1][i])
            write(1, &av[1][i++], 1); // print each char of first argument
    write(1, "\n", 1);
}
```

aff_last_param

- **File:** aff_last_param.c | **Allowed:** write

Subject

Write a program that takes strings as arguments, and displays its last argument followed by a newline.

If the number of arguments is less than 1, the program displays a newline.

Examples:

```
$> ./aff_last_param "zaz" "mange" "des" "chats"
chats
$> ./aff_last_param "j'aime le savon"
j'aime le savon
$> ./aff_last_param
$
```

- **Approach:** argv[argc - 1] is always the last argument. Print it if argc > 1.

```
#include <unistd.h>
int main(int ac, char **av)
{
    int i = 0;
    if (ac > 1) // at least one argument
        while (av[ac - 1][i])
            write(1, &av[ac - 1][i++], 1); // print each char of last argument
    write(1, "\n", 1);
}
```

aff_z

- **File:** aff_z.c | **Allowed:** write

Subject

Write a program that takes a string, and displays the character 'z' followed by a newline.

Example:

```
$> ./aff_z "abc" | cat -e
z$
$> ./aff_z | cat -e
```

z\$

- **Approach:** Always print z\n, regardless of arguments.

```
#include <unistd.h>
int main(void)
{
    write(1, "z\n", 2);           // always print 'z' and newline
}
```

first_word

- **File:** first_word.c | **Allowed:** write

Subject

Write a program that takes a string and displays its first word, followed by a newline.

A word is a section of string delimited by spaces/tabs or by the start/end of the string.

If the number of parameters is not 1, or if there are no words, simply display a newline.

Examples:

```
$> ./first_word "FOR PONY" | cat -e
FOR$
$> ./first_word "this      ...   is sparta, then again, maybe   not" | cat -e
this$
$> ./first_word "   " | cat -e
$
$> ./first_word "a" "b" | cat -e
$
$> ./first_word "   lorem,ipsum   " | cat -e
lorem,ipsum$
```

- **Approach:** Skip leading spaces/tabs, then print characters until the next space/tab or end of string.

```
#include <unistd.h>
int main(int ac, char **av)
{
    int i = 0;
    if (ac == 2)
    {
        while (av[1][i] == ' ' || av[1][i] == '\t') // skip leading whitespace
            i++;
        while (av[1][i] && av[1][i] != ' ' && av[1][i] != '\t') // print first word
            write(1, &av[1][i++], 1);
    }
    write(1, "\n", 1);
}
```

fizzbuzz

- **File:** fizzbuzz.c | **Allowed:** write

Subject

Write a program that prints the numbers from 1 to 100, each separated by a newline.

If the number is a multiple of 3, it prints 'fizz' instead.

If the number is a multiple of 5, it prints 'buzz' instead.

If the number is both a multiple of 3 and a multiple of 5, it prints 'fizzbuzz' instead.

Example:

```
$>./fizzbuzz
```

```
1
```

```
2
```

```
fizz
```

```
4
```

```
buzz
```

```
fizz
```

```
[...]
```

- **Approach:** Check divisibility by 15 first (to catch fizzbuzz), then 3, then 5. Use a helper to print numbers since only write is allowed.

```
#include <unistd.h>
void putnbr(int n)
{
    if (n >= 10) putnbr(n / 10);           // print leading digits recursively
    char c = n % 10 + '0';
    write(1, &c, 1);
}
int main(void)
{
    int i = 1;
    while (i <= 100)
    {
        if (i % 15 == 0) write(1, "fizzbuzz", 8); // divisible by both 3 and 5
        else if (i % 3 == 0) write(1, "fizz", 4); // divisible by 3 only
        else if (i % 5 == 0) write(1, "buzz", 4); // divisible by 5 only
        else putnbr(i);                          // not divisible by 3 or 5
        write(1, "\n", 1);
        i++;
    }
}
```

ft_putstr

- **File:** ft_putstr.c | **Allowed:** write

Subject

Write a function that displays a string on the standard output.

The pointer passed to the function contains the address of the string's first character.

Your function must be declared as follows:

```
void ft_putstr(char *str);
```

- **Approach:** Loop over each character and write it one by one until \0.

```
#include <unistd.h>
void ft_putstr(char *str)
{
    int i = 0;
    while (str[i])
        write(1, &str[i++], 1);           // write one char at a time
}
```

ft_strcpy

- **File:** ft_strcpy.c | **Allowed:** none

Subject

Reproduce the behavior of the function strcpy (man strcpy).

Your function must be declared as follows:

```
char *ft_strcpy(char *s1, char *s2);
```

- **Approach:** Copy each character from s2 into s1, including the null terminator. Return s1.

```
char *ft_strcpy(char *s1, char *s2)
{
    int i = 0;
    while (s2[i])
        s1[i] = s2[i++];           // copy char by char
    s1[i] = '\0';                  // null-terminate destination
    return (s1);
}
```

ft_strlen

- **File:** ft_strlen.c | **Allowed:** none

Subject

Write a function that returns the length of a string.

Your function must be declared as follows:

```
int ft_strlen(char *str);
```

- **Approach:** Count characters until \0. Return the count.

```
int ft_strlen(char *str)
{
    int i = 0;
    while (str[i])
        i++;                       // count until null terminator
    return (i);
}
```

ft_swap

- **File:** ft_swap.c | **Allowed:** none

Subject

Write a function that swaps the contents of two integers the addresses of which are passed as parameters.

Your function must be declared as follows:

```
void ft_swap(int *a, int *b);
```

- **Approach:** Use a temporary variable to exchange the values at the two addresses.

```
void ft_swap(int *a, int *b)
{
    int tmp = *a;           // save value of a
    *a = *b;                // overwrite a with b
    *b = tmp;               // overwrite b with saved a
}
```

maff_alpha

- **File:** maff_alpha.c | **Allowed:** write

Subject

Write a program that displays the alphabet, with even letters in uppercase, and odd letters in lowercase, followed by a newline.

Example:

```
$> ./maff_alpha | cat -e
aBcDeFgHiJkLmNoPqRsTuVwXyZ$
```

- **Approach:** The output is fixed — just hardcode the string.

```
#include <unistd.h>
int main(void)
{
    write(1, "aBcDeFgHiJkLmNoPqRsTuVwXyZ\n", 27); // hardcoded result
}
```

maff_revalpha

- **File:** maff_revalpha.c | **Allowed:** write

Subject

Write a program that displays the alphabet in reverse order, with even letters in uppercase, and odd letters in lowercase, followed by a newline.

Example:

```
$> ./maff_revalpha | cat -e
zYxWvUtSrQpOnMLKjIhGfEdCbA$
```

- **Approach:** The output is fixed — just hardcode the string.

```
#include <unistd.h>
int main(void)
{
    write(1, "zYxWvUtSrQpOnMLKjIhGfEdCbA\n", 27); // hardcoded result
}
```

repeat_alpha

- **File:** repeat_alpha.c | **Allowed:** write

Subject

Write a program called repeat_alpha that takes a string and display it repeating each alphabetical character as many times as its alphabetical index, followed by a newline.

'a' becomes 'a', 'b' becomes 'bb', 'e' becomes 'eeee', etc...

Case remains unchanged.

If the number of arguments is not 1, just display a newline.

Examples:

```
$>./repeat_alpha "abc"
abbccc
$>./repeat_alpha "Alex." | cat -e
Alllllllllllleeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeee.$
$>./repeat_alpha 'abacadaba 42!' | cat -e
abbaccaddddabba 42!$
$>./repeat_alpha | cat -e
$
```

- **Approach:** For each character, compute its alphabetical index (A=1, B=2...), then write it that many times. Non-letters are written once.

```
#include <unistd.h>
int main(int ac, char **av)
{
    int i = 0;
    int k;
    if (ac == 2)
        while (av[1][i])
        {
            if (av[1][i] >= 'A' && av[1][i] <= 'Z')
                k = av[1][i] - 'A' + 1;           // uppercase: A=1, B=2, ...
            else if (av[1][i] >= 'a' && av[1][i] <= 'z')
                k = av[1][i] - 'a' + 1;           // lowercase: a=1, b=2, ...
            else
                k = 1;                             // non-letter: write once
            while (k--)
                write(1, &av[1][i], 1);           // repeat k times
            i++;
        }
    write(1, "\n", 1);
}
```

rev_print

- **File:** rev_print.c | **Allowed:** write

Subject

Write a program that takes a string, and displays the string in reverse followed by a newline.

If the number of parameters is not 1, the program displays a newline.

Examples:

```
$> ./rev_print "zaz" | cat -e
zaz$
$> ./rev_print "dub0 a P0IL" | cat -e
LI0P a 0bud$
$> ./rev_print | cat -e
$
```

- **Approach:** Walk to the end of the string, then print backwards by decrementing the index.

```
#include <unistd.h>
int main(int ac, char **av)
{
    int i = 0;
    if (ac == 2)
    {
        while (av[1][i]) i++;           // move i to end of string
        while (i--) write(1, &av[1][i], 1); // print backwards
    }
    write(1, "\n", 1);
}
```

rot_13

- **File:** rot_13.c | **Allowed:** write

Subject

Write a program that takes a string and displays it, replacing each of its letters by the letter 13 spaces ahead in alphabetical order.

'z' becomes 'm' and 'Z' becomes 'M'. Case remains unaffected.

The output will be followed by a newline.

If the number of arguments is not 1, the program displays a newline.

Example:

```
$> ./rot_13 "abc"
nop
$> ./rot_13 "My horse is Amazing." | cat -e
Zl ubefr vf Nznmvat.$
$> ./rot_13 "AkjhZ zLKIJz , 23y " | cat -e
NxwuM mYXVWm , 23l $
```

- **Approach:** Letters a-m shift +13, letters n-z shift -13. Same for uppercase. Non-letters are unchanged.

```
#include <unistd.h>
int main(int ac, char **av)
{
    int i = 0;
    if (ac == 2)
        while (av[1][i])
        {
            if ((av[1][i] >= 'a' && av[1][i] <= 'm') || (av[1][i] >= 'A' && av[1][i] <= 'M'))
                av[1][i] += 13;           // first half of alphabet: shift forward
            else if ((av[1][i] >= 'n' && av[1][i] <= 'z') || (av[1][i] >= 'N' && av[1][i] <= 'Z'))
                av[1][i] -= 13;           // second half of alphabet: shift back
        }
    write(1, av[1], strlen(av[1]));
    write(1, "\n", 1);
}
```

```

        write(1, &av[1][i], 1);
        i++;
    }
    write(1, "\n", 1);
}

```

rotone

- **File:** rotone.c | **Allowed:** write

Subject

Write a program that takes a string and displays it, replacing each of its letters by the next one in alphabetical order.

'z' becomes 'a' and 'Z' becomes 'A'. Case remains unaffected.

The output will be followed by a \n.

If the number of arguments is not 1, the program displays \n.

Example:

```
$>./rotone "abc"
```

```
bcd
```

```
$>./rotone "Les stagiaires du staff ne sentent pas toujours tres bon." | cat -e
Mft tubhjbjsft ev tubgg of tfoufou qbt upvkpvst usft cpo.$
```

```
$>./rotone "AkjhZ zLKIJz , 23y " | cat -e
```

```
BlkiA aMLJKa , 23z $
```

- **Approach:** Add 1 to each letter. Wrap z→a and Z→A. Non-letters are unchanged.

```

#include <unistd.h>
int main(int ac, char **av)
{
    int i = 0;
    if (ac == 2)
        while (av[1][i])
        {
            if ((av[1][i] >= 'a' && av[1][i] <= 'y') || (av[1][i] >= 'A' && av[1][i]
                <= 'Y'))
                av[1][i] += 1; // shift letter forward by one
            else if (av[1][i] == 'z') av[1][i] = 'a'; // wrap lowercase z
            else if (av[1][i] == 'Z') av[1][i] = 'A'; // wrap uppercase Z
            write(1, &av[1][i], 1);
            i++;
        }
    write(1, "\n", 1);
}

```

search_and_replace

- **File:** search_and_replace.c | **Allowed:** write, exit

Subject

Write a program called search_and_replace that takes 3 arguments, the first arguments is a string in which to replace a letter (2nd argument) by another one (3rd argument).

If the number of arguments is not 3, just display a newline.

If the second argument is not contained in the first one (the string) then the program simply rewrites the string followed by a newline.

Examples:

```
$>./search_and_replace "Papache est un sabre" "a" "o"
Popoche est un sobre
$>./search_and_replace "zaz" "art" "zul" | cat -e
$
$>./search_and_replace "zaz" "r" "u" | cat -e
zaz$
$>./search_and_replace "jacob" "a" "b" "c" "e" | cat -e
$
$>./search_and_replace "ZoZ eT Dovid oiME le METol." "o" "a" | cat -e
ZaZ eT David aiME le METal.$
```

- **Approach:** Require exactly 4 args and single-char argv[2]/argv[3]. Scan the string and replace every match.

```
#include <unistd.h>
int main(int ac, char **av)
{
    int i = 0;
    // need exactly 4 args and both search/replace must be single characters
    if (ac == 4 && av[2][1] == '\0' && av[3][1] == '\0')
    {
        while (av[1][i])
        {
            if (av[1][i] == av[2][0])    // replace matching character
                av[1][i] = av[3][0];
            write(1, &av[1][i], 1);
            i++;
        }
        write(1, "\n", 1);
    }
}
```

ulstr

- **File:** ulstr.c | **Allowed:** write

Subject

Write a program that takes a string and reverses the case of all its letters. Other characters remain unchanged.

You must display the result followed by a '\n'.

If the number of arguments is not 1, the program displays '\n'.

Examples:

```
$>./ulstr "L'eSPrit nE peUt pLUs pRogResSer s'Il staGne et sI peRsIsTent VAnIte et auto-justification." | cat -e
l'EspRiT Ne PEuT PLuS PRoGrESsER S'iL STAgNE ET Si PErSiStENT vaNiTE ET AUTO-JUSTIFICATION.$
$>./ulstr | cat -e
$
```

- **Approach:** Subtract 32 for lowercase→uppercase, add 32 for uppercase→lowercase. Non-

letters unchanged.

```
#include <unistd.h>
int main(int ac, char **av)
{
    int i = 0;
    if (ac == 2)
        while (av[1][i])
        {
            if (av[1][i] >= 'a' && av[1][i] <= 'z')    av[1][i] -= 32; // lower →
                ↪ upper
            else if (av[1][i] >= 'A' && av[1][i] <= 'Z') av[1][i] += 32; // upper →
                ↪ lower
            write(1, &av[1][i], 1);
            i++;
        }
    write(1, "\n", 1);
}
```

Level 1

alpha_mirror

- **File:** alpha_mirror.c | **Allowed:** write

Subject

Write a program called alpha_mirror that takes a string and displays this string after replacing each alphabetical character by the opposite alphabetical character, followed by a newline.

'a' becomes 'z', 'Z' becomes 'A'
'd' becomes 'w', 'M' becomes 'N'

Case is not changed.

If the number of arguments is not 1, display only a newline.

Examples:

```
$>./alpha_mirror "abc"
```

```
zyx
```

```
$>./alpha_mirror "My horse is Amazing." | cat -e
```

```
Nb slihv rh Znzarmt.$
```

```
$>./alpha_mirror | cat -e
```

```
$
```

- **Approach:** Mirror each letter: 'a' + 'z' - c maps a↔z, b↔y, etc. Same formula works for uppercase with 'A' and 'Z'.

```
#include <unistd.h>
int main(int ac, char **av)
{
    int i = 0;
    if (ac == 2)
        while (av[1][i])
        {
            if (av[1][i] >= 'a' && av[1][i] <= 'z')
                av[1][i] = 'a' + 'z' - av[1][i]; // mirror lowercase: a↔z, b↔y...
            else if (av[1][i] >= 'A' && av[1][i] <= 'Z')
                av[1][i] = 'A' + 'Z' - av[1][i]; // mirror uppercase: A↔Z, B↔Y...
```

```

        av[1][i] = 'A' + 'Z' - av[1][i]; // mirror uppercase: A↔Z, B↔Y...
        write(1, &av[1][i], 1);
        i++;
    }
    write(1, "\n", 1);
}

```

camel_to_snake

- **File:** camel_to_snake.c | **Allowed:** malloc, free, realloc, write

Subject

Write a program that takes a single string in lowerCamelCase format and converts it into a string in snake_case format.

A lowerCamelCase string is a string where each word begins with a capital letter except for the first one.

A snake_case string is a string where each word is in lower case, separated by an underscore "_".

Examples:

```

$>./camel_to_snake "hereIsACamelCaseWord"
here_is_a_camel_case_word
$>./camel_to_snake "helloWorld" | cat -e
hello_world$
$>./camel_to_snake | cat -e
$

```

- **Approach:** When an uppercase letter is found, print _ first, then print the letter lowercased.

```

#include <unistd.h>
int main(int ac, char **av)
{
    int i = 0;
    if (ac == 2)
        while (av[1][i])
        {
            if (av[1][i] >= 'A' && av[1][i] <= 'Z')
            {
                write(1, "_", 1); // insert underscore before capital
                av[1][i] += 32;    // convert to lowercase
            }
            write(1, &av[1][i], 1);
            i++;
        }
    write(1, "\n", 1);
}

```

do_op

- **File:** do_op.c | **Allowed:** atoi, printf, write

Subject

Write a program that takes three strings:

- The first and the third one are representations of base-10 signed integers

that fit in an int.

- The second one is an arithmetic operator chosen from: + - * / %

The program must display the result of the requested arithmetic operation, followed by a newline. If the number of parameters is not 3, the program just displays a newline.

Examples:

```
$> ./do_op "123" "*" 456 | cat -e
56088$
$> ./do_op "9828" "/" 234 | cat -e
42$
$> ./do_op "1" "+" "-43" | cat -e
-42$
$> ./do_op | cat -e
$
```

- **Approach:** Convert argv[1] and argv[3] with atoi. Check argv[2][0] for the operator. Print with printf.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
int main(int ac, char **av)
{
    if (ac == 4)
    {
        int a = atoi(av[1]);           // first operand
        int b = atoi(av[3]);           // second operand
        int r = 0;
        if (av[2][0] == '+') r = a + b;
        else if (av[2][0] == '-') r = a - b;
        else if (av[2][0] == '*') r = a * b;
        else if (av[2][0] == '/') r = a / b;
        else if (av[2][0] == '%') r = a % b;
        printf("%d\n", r);
    }
    else
        write(1, "\n", 1);             // wrong number of arguments
}
```

ft_atoi

- **File:** ft_atoi.c | **Allowed:** none

Subject

Write a function that converts the string argument str to an integer (type int) and returns it.

It works much like the standard atoi(const char *str) function, see the man.

Your function must be declared as follows:

```
int ft_atoi(const char *str);
```

- **Approach:** Handle optional +/- sign, then accumulate digits. Return result multiplied by sign.

```

int ft_atoi(const char *str)
{
    int n = 0;
    int sign = 1;
    int i = 0;
    if (str[i] == '-') { sign = -1; i++; } // negative sign
    else if (str[i] == '+') i++;           // positive sign (skip)
    while (str[i] >= '0' && str[i] <= '9') // build number digit by digit
        n = n * 10 + (str[i++] - '0');
    return (n * sign);
}

```

ft_strchr

- **File:** ft_strchr.c | **Allowed:** none

Subject

Reproduce the behavior of the function strchr (man strchr).

The function must be declared as follows:

```
char *ft_strchr(const char *s, int c);
```

- **Approach:** Scan the string for c. If found, return a pointer to it. Also check for \0 after the loop (in case c == 0).

```

char *ft_strchr(const char *s, int c)
{
    while (*s)
    {
        if (*s == (char)c)
            return ((char *)s); // found: return pointer to match
        s++;
    }
    if ((char)c == '\0')
        return ((char *)s); // c is '\0': return pointer to null
    // c is '\0': return pointer to null
    return (NULL);
}

```

ft_strcmp

- **File:** ft_strcmp.c | **Allowed:** none

Subject

Reproduce the behavior of the function strcmp (man strcmp).

Your function must be declared as follows:

```
int ft_strcmp(char *s1, char *s2);
```

- **Approach:** Advance while characters are equal and non-null. Return the difference of the first differing characters.

```

int ft_strcmp(char *s1, char *s2)
{

```

```

int i = 0;
while (s1[i] && s1[i] == s2[i])    // skip matching characters
    i++;
return ((unsigned char)s1[i] - (unsigned char)s2[i]); // difference at mismatch
}

```

ft_strcspn

- **File:** ft_strcspn.c | **Allowed:** none

Subject

Reproduce exactly the behavior of the function strcspn (man strcspn).

The function should be prototyped as follows:

```
size_t ft_strcspn(const char *s, const char *reject);
```

- **Approach:** For each character in s, check if it appears in reject. Return the index of the first match.

```

#include <stddef.h>
size_t ft_strcspn(const char *s, const char *reject)
{
    size_t i = 0;
    while (s[i])
    {
        size_t k = 0;
        while (reject[k])
        {
            if (s[i] == reject[k]) return (i); // found a rejected char: stop
            k++;
        }
        i++;
    }
    return (i); // no rejected char found: return full length
}

```

ft_strdup

- **File:** ft_strdup.c | **Allowed:** malloc

Subject

Reproduce the behavior of the function strdup (man strdup).

Your function must be declared as follows:

```
char *ft_strdup(char *src);
```

- **Approach:** Measure the string length, malloc length + 1, then copy character by character including \0.

```

#include <stdlib.h>
char *ft_strdup(char *src)
{
    int i = 0;
    while (src[i]) i++; // get string length
}

```

```

char *dst = malloc(i + 1);           // allocate length + null terminator
if (!dst) return (NULL);
int j = 0;
while (src[j]) { dst[j] = src[j]; j++; } // copy characters
dst[j] = '\0';
return (dst);
}

```

ft_strpbrk

- **File:** ft_strpbrk.c | **Allowed:** none

Subject

Reproduce the behavior of the function strpbrk (man strpbrk).

The function must be declared as follows:

```
char *ft_strpbrk(const char *s1, const char *s2);
```

- **Approach:** For each character in s1, scan s2 for a match. Return pointer to the first match, or NULL.

```

char *ft_strpbrk(const char *s1, const char *s2)
{
    while (*s1)
    {
        int i = 0;
        while (s2[i])
            if (*s1 == s2[i++]) return ((char *)s1); // found char from s2 in s1
        s1++;
    }
    return (NULL); // no character from s2 found in s1
}

```

ft_strrev

- **File:** ft_strrev.c | **Allowed:** none

Subject

Write a function that reverses (in-place) a string.

It must return its parameter.

Your function must be declared as follows:

```
char *ft_strrev(char *str);
```

- **Approach:** Swap characters from both ends moving inward until the middle.

```

char *ft_strrev(char *str)
{
    int len = 0;
    while (str[len]) len++; // get string length
    int i = 0;
    while (i < len / 2)
    {

```

```

        char tmp = str[i];                // swap characters from both ends
        str[i] = str[len - 1 - i];
        str[len - 1 - i] = tmp;
        i++;
    }
    return (str);
}

```

ft_strspn

- **File:** ft_strspn.c | **Allowed:** none

Subject

Reproduce the behavior of the function strspn (man strspn).

The function must be declared as follows:

```
size_t    ft_strspn(const char *s, const char *accept);
```

- **Approach:** For each character in s, check if it exists in accept. Stop and return the index when a character is not found.

```

#include <stddef.h>
size_t ft_strspn(const char *s, const char *accept)
{
    size_t i = 0;
    while (s[i])
    {
        int j = 0;
        while (accept[j] && accept[j] != s[i]) j++; // search s[i] in accept
        if (!accept[j]) return (i);                // s[i] not in accept: stop
        i++;
    }
    return (i);
}

```

inter

- **File:** inter.c | **Allowed:** write

Subject

Write a program that takes two strings and displays, without doubles, the characters that appear in both strings, in the order they appear in the first one.

The display will be followed by a \n.

If the number of arguments is not 2, the program displays \n.

Examples:

```

$>./inter "padinton" "paqefwtdjetyiytjneytjoeyjnejejj" | cat -e
padinto$
$>./inter ddf6vewg64f gtwthgdwthdwfteewhrtag6h4ffdhdsd | cat -e
df6ewg4$
$>./inter "rien" "cette phrase ne cache rien" | cat -e
rien$
$>./inter | cat -e

```

\$

- **Approach:** For each character in argv[1], check it exists in argv[2] AND hasn't already been printed (by checking earlier positions in argv[1]).

```
#include <unistd.h>
int main(int ac, char **av)
{
    int i = 0;
    if (ac == 3)
        while (av[1][i])
        {
            int k = 0;
            while (av[2][k] && av[2][k] != av[1][i]) k++; // search in s2
            if (av[2][k]) // char exists in s2
            {
                int dup = 0;
                int l = 0;
                while (l < i) { if (av[1][l] == av[1][i]) { dup = 1; break; } l++; }
                // check for duplicate in s1
                if (!dup) write(1, &av[1][i], 1); // print only if not already printed
            }
            i++;
        }
    write(1, "\n", 1);
}
```

is_power_of_2

- **File:** is_power_of_2.c | **Allowed:** none

Subject

Write a function that determines if a given number is a power of 2.

This function returns 1 if the given number is a power of 2, otherwise it returns 0.

Your function must be declared as follows:

```
int is_power_of_2(unsigned int n);
```

- **Approach:** A power of 2 has exactly one bit set. The bit trick $n \& (n - 1)$ clears the lowest set bit — if the result is 0, n was a power of 2. Handle $n == 0$ separately.

```
int is_power_of_2(unsigned int n)
{
    if (n == 0) return (0); // 0 is not a power of 2
    return ((n & (n - 1)) == 0); // true if only one bit is set
}
```

last_word

- **File:** last_word.c | **Allowed:** write

Subject

Write a program that takes a string and displays its last word followed by a `\n`.

A word is a section of string delimited by spaces/tabs or by the start/end of the string.

If the number of parameters is not 1, or there are no words, display a newline.

Example:

```
$> ./last_word "FOR PONY" | cat -e
PONY$
$> ./last_word "this      ...      is sparta, then again, maybe      not" | cat -e
not$
$> ./last_word "  " | cat -e
$
$> ./last_word "a" "b" | cat -e
$
$> ./last_word "  lorem,ipsum  " | cat -e
lorem,ipsum$
```

- **Approach:** Scan the string tracking the start of each new word. After the loop, start points to the last word's beginning. Print from there while non-whitespace.

```
#include <unistd.h>
int main(int ac, char **av)
{
    int i = 0;
    int start = 0;
    if (ac == 2)
    {
        while (av[1][i])
        {
            // update start whenever a new word begins
            if (av[1][i] != ' ' && av[1][i] != '\t' &&
                (i == 0 || av[1][i - 1] == ' ' || av[1][i - 1] == '\t'))
                start = i;
            i++;
        }
        // print the last word if start points to a non-whitespace character
        if (av[1][start] && av[1][start] != ' ' && av[1][start] != '\t')
            while (av[1][start] && av[1][start] != ' ' && av[1][start] != '\t')
                write(1, &av[1][start++], 1);
    }
    write(1, "\n", 1);
}
```

max

- **File:** max.c | **Allowed:** none

Subject

Write the following function:

```
int max(int* tab, unsigned int len);
```

The first parameter is an array of int, the second is the number of elements in the array.

The function returns the largest number found in the array.
If the array is empty, the function returns 0.

- **Approach:** Start with `tab[0]` as the candidate max, then update whenever a larger value is found. Return 0 for an empty array.

```
int max(int *tab, unsigned int len)
{
    if (len == 0) return (0);           // empty array
    int m = tab[0];                     // assume first element is max
    unsigned int i = 1;
    while (i < len)
    {
        if (tab[i] > m) m = tab[i];     // update max if larger found
        i++;
    }
    return (m);
}
```

print_bits

- **File:** `print_bits.c` | **Allowed:** write

Subject

Write a function that takes a byte, and prints it in binary WITHOUT A NEWLINE AT THE END.

Your function must be declared as follows:

```
void print_bits(unsigned char octet);
```

Example, if you pass 2 to `print_bits`, it will print "00000010"

- **Approach:** Iterate from bit 7 down to bit 0. Extract each bit with `(octet >> i) & 1` and print '0' or '1'.

```
#include <unistd.h>
void print_bits(unsigned char octet)
{
    int i = 8;
    while (i-->0)
    {
        char b = ((octet >> i) & 1) + '0'; // extract bit i, convert to char
        write(1, &b, 1);
    }
}
```

reverse_bits

- **File:** `reverse_bits.c` | **Allowed:** none

Subject

Write a function that takes a byte, reverses it, bit by bit (like the example) and returns the result.

Your function must be declared as follows:

```
unsigned char reverse_bits(unsigned char octet);
```

Example:

1 byte

```
0010 0110
  ||
  \/  
0110 0100
```

- **Approach:** Build the result by extracting the LSB of octet each iteration and left-shifting the result.

```
unsigned char reverse_bits(unsigned char octet)
{
    unsigned char res = 0;
    int i = 8;
    while (i-- > 0)
    {
        res = res * 2 + (octet % 2);    // shift result left, add LSB of octet
        octet /= 2;                     // shift octet right
    }
    return (res);
}
```

snake_to_camel

- **File:** snake_to_camel.c | **Allowed:** malloc, free, realloc, write

Subject

Write a program that takes a single string in snake_case format and converts it into a string in lowerCamelCase format.

A snake_case string is a string where each word is in lower case, separated by an underscore "_".

A lowerCamelCase string is a string where each word begins with a capital letter except for the first one.

Examples:

```
$> ./snake_to_camel "here_is_a_snake_case_word"
hereIsASnakeCaseWord
$> ./snake_to_camel "hello_world" | cat -e
helloWorld$
$> ./snake_to_camel | cat -e
$
```

- **Approach:** When an _ is found, skip it and uppercase the next character. Otherwise print as-is.

```
#include <unistd.h>
int main(int ac, char **av)
{
    int i = 0;
    if (ac == 2)
        while (av[1][i])
        {
            if (av[1][i] == '_')
            {
                i++;
                av[1][i] -= 32;    // skip the underscore
                                // uppercase the following letter
            }
            i++;
        }
}
```

```

        write(1, &av[1][i], 1);
        i++;
    }
    write(1, "\n", 1);
}

```

swap_bits

- **File:** swap_bits.c | **Allowed:** none

Subject

Write a function that takes a byte, swaps its halves (like the example) and returns the result.

Your function must be declared as follows:

```
unsigned char    swap_bits(unsigned char octet);
```

Example:

1 byte

```

0100 | 0001
   \ /
   / \
0001 | 0100

```

- **Approach:** Shift the high nibble right by 4 and OR it with the low nibble shifted left by 4.

```

unsigned char swap_bits(unsigned char octet)
{
    return ((octet >> 4) | (octet << 4)); // swap high and low nibbles
}

```

union

- **File:** union.c | **Allowed:** write

Subject

Write a program that takes two strings and displays, without doubles, the characters that appear in either one of the strings.

The display will be in the order characters appear in the command line, and will be followed by a \n.

If the number of arguments is not 2, the program displays \n.

Example:

```

$>./union zpadinton "paqefwtdjetiyjtjneytjoeyjnejejj" | cat -e
zpadintoqefwjy$
$>./union ddf6vewg64f gtwthgdwthdwfteewhrtag6h4ffdhdsd | cat -e
df6vewg4thras$
$>./union "rien" "cette phrase ne cache rien" | cat -e
rienct phas$
$>./union | cat -e
$

```

- **Approach:** Use a `seen[256]` boolean array indexed by ASCII value to track already-printed characters. Process `argv[1]` then `argv[2]`.

```
#include <unistd.h>
static char seen[256] = {0};           // tracks which chars have been printed
int main(int ac, char **av)
{
    int i = 0;
    if (ac == 3)
    {
        while (av[1][i])               // process first string
        {
            if (!seen[(unsigned char)av[1][i]])
            {
                seen[(unsigned char)av[1][i]] = 1;
                write(1, &av[1][i], 1);
            }
            i++;
        }
        i = 0;
        while (av[2][i])               // process second string
        {
            if (!seen[(unsigned char)av[2][i]])
            {
                seen[(unsigned char)av[2][i]] = 1;
                write(1, &av[2][i], 1);
            }
            i++;
        }
        write(1, "\n", 1);
    }
}
```

wdmatch

- **File:** `wdmatch.c` | **Allowed:** write

Subject

Write a program that takes two strings and checks whether it's possible to write the first string with characters from the second string, while respecting the order in which these characters appear in the second string.

If it's possible, the program displays the string, followed by a `\n`, otherwise it simply displays a `\n`.

If the number of arguments is not 2, the program displays a `\n`.

Examples:

```
$>./wdmatch "faya" "fgvvfdxcacpolhyghbreda" | cat -e
```

```
faya$
```

```
$>./wdmatch "faya" "fgvvfdxcacpolhyghbred" | cat -e
```

```
$
```

```
$>./wdmatch "quarante deux" "qfqfsudf arzgsayns tsregfdgs sjytdkuoixq " | cat -e
quarante deux$
```

```
$>./wdmatch "error" "rrrrrrfiiljdfxjyuifrrvcoojh" | cat -e
```

```
$
```

- **Approach:** Walk `argv[2]` character by character. Each time a char matches the current position

in argv[1], advance the argv[1] index. If we reach the end of argv[1], all chars were found in order.

```
#include <unistd.h>
int main(int ac, char **av)
{
    int i = 0;                // index in argv[1]
    int j = 0;                // index in argv[2]
    if (ac == 3)
    {
        while (av[2][j])
            if (av[2][j++] == av[1][i]) i++; // advance i when match found
        if (!av[1][i])                // all chars of argv[1] were matched
        {
            int k = 0;
            while (av[1][k]) write(1, &av[1][k++], 1);
        }
    }
    write(1, "\n", 1);
}
```

Level 2

add_prime_sum

- **File:** add_prime_sum.c | **Allowed:** write, exit

Subject

Write a program that takes a positive integer as argument and displays the sum of all prime numbers inferior or equal to it followed by a newline.

If the number of arguments is not 1, or the argument is not a positive number, just display 0 followed by a newline.

Examples:

```
$>./add_prime_sum 5
10
$>./add_prime_sum 7 | cat -e
17$
$>./add_prime_sum | cat -e
0$
```

- **Approach:** For each number 2...n, check if it's prime (no divisor from 2 to n-1), and accumulate the sum. Print 0 if no valid argument.

```
#include <unistd.h>
void putnbr(int n)
{
    if (n >= 10) putnbr(n / 10); // print leading digits recursively
    char c = n % 10 + '0';
    write(1, &c, 1);
}
int is_prime(int n)
{
    int i = 2;
    while (i < n) { if (n % i == 0) return (0); i++; } // any divisor found → not
    ↪ prime
}
```

```

    return (n >= 2);                // 0 and 1 are not prime
}
int ft_atoi(char *s)
{
    int n = 0;
    while (*s >= '0' && *s <= '9') n = n * 10 + (*s++ - '0');
    return (n);
}
int main(int ac, char **av)
{
    int n = (ac == 2) ? ft_atoi(av[1]) : 0; // 0 if no argument
    int sum = 0;
    int i = 2;
    while (i <= n) { if (is_prime(i)) sum += i; i++; } // accumulate prime sum
    putnbr(sum);
    write(1, "\n", 1);
}

```

epur_str

- **File:** epur_str.c | **Allowed:** write

Subject

Write a program that takes a string, and displays this string with exactly one space between words, with no spaces or tabs either at the beginning or the end, followed by a \n.

A "word" is defined as a part of a string delimited either by spaces/tabs, or by the start/end of the string.

If the number of arguments is not 1, or if there are no words to display, the program displays \n.

Example:

```

$> ./epur_str "See? It's easy to print the same thing" | cat -e
See? It's easy to print the same thing$
$> ./epur_str " this      time it      will      be      more complex . " | cat -e
this time it will be more complex .$
$> ./epur_str "No S*** Sherlock..." "nAw S*** ShErLaWQ..." | cat -e
$
$> ./epur_str "" | cat -e
$

```

- **Approach:** Skip whitespace, print a space before each word except the first, then print the word character by character.

```

#include <unistd.h>
int main(int ac, char **av)
{
    int i = 0;
    int first = 1;                // tracks if it's the first word
    if (ac == 2)
        while (av[1][i])
        {
            while (av[1][i] == ' ' || av[1][i] == '\t') i++; // skip whitespace
            if (!av[1][i]) break;

```

```

        if (!first) write(1, " ", 1); // print separator before non-first words
        first = 0;
        while (av[1][i] && av[1][i] != ' ' && av[1][i] != '\t')
            write(1, &av[1][i++], 1); // print word
    }
    write(1, "\n", 1);
}

```

expand_str

- **File:** expand_str.c | **Allowed:** write

Subject

Write a program that takes a string and displays it with exactly three spaces between each word, with no spaces or tabs either at the beginning or the end, followed by a newline.

A word is a section of string delimited either by spaces/tabs, or by the start/end of the string.

If the number of parameters is not 1, or if there are no words, simply display a newline.

Examples:

```

$> ./expand_str "See? It's easy to print the same thing" | cat -e
See?  It's  easy  to  print  the  same  thing$
$> ./expand_str "this time it will be more complex " | cat -e
this  time  it  will  be  more  complex$
$> ./expand_str "No S*** Sherlock..." "nAw S*** ShErLaWQ..." | cat -e
$
$> ./expand_str "" | cat -e
$

```

- **Approach:** Same as epur_str but print " " (3 spaces) between words instead of one.

```

#include <unistd.h>
int main(int ac, char **av)
{
    int i = 0;
    int first = 1; // tracks if it's the first word
    if (ac == 2)
        while (av[1][i])
        {
            while (av[1][i] == ' ' || av[1][i] == '\t') i++; // skip whitespace
            if (!av[1][i]) break;
            if (!first) write(1, "   ", 3); // 3-space separator between words
            first = 0;
            while (av[1][i] && av[1][i] != ' ' && av[1][i] != '\t')
                write(1, &av[1][i++], 1); // print word
        }
    write(1, "\n", 1);
}

```

ft_atoi_base

- **File:** ft_atoi_base.c | **Allowed:** none

Subject

Write a function that converts the string argument `str` (base $N \leq 16$) to an integer (base 10) and returns it.

The characters recognized in the input are: 0123456789abcdef
Those are, of course, to be trimmed according to the requested base.

Uppercase letters must also be recognized: "12fdb3" is the same as "12FDB3".

Minus signs ('-') are interpreted only if they are the first character of the string.

Your function must be declared as follows:

```
int ft_atoi_base(const char *str, int str_base);
```

- **Approach:** Handle optional - sign. Convert each character to its digit value (0-15). Stop when a character is not valid for the given base.

```
int ft_atoi_base(const char *str, int base)
{
    int sign = 1;
    int result = 0;
    if (*str == '-') { sign = -1; str++; } // handle negative sign
    while (*str)
    {
        // normalize uppercase to lowercase
        char c = (*str >= 'A' && *str <= 'Z') ? *str + 32 : *str;
        int d;
        if (c >= '0' && c <= '9') d = c - '0'; // digit 0-9
        else if (c >= 'a' && c <= 'f') d = 10 + c - 'a'; // digit a-f
        else break; // invalid char: stop
        if (d >= base) break; // digit out of range for this base
        result = result * base + d * sign;
        str++;
    }
    return (result);
}
```

ft_list_size

- **File:** ft_list_size.c | **Allowed:** none

Subject

Write a function that returns the number of elements in the linked list that's passed to it.

It must be declared as follows:

```
int ft_list_size(t_list *begin_list);
```

You must use the following structure in your program ft_list_size.c:

```
typedef struct s_list
{
    struct s_list *next;
```

```

    void          *data;
}
t_list;

```

- **Approach:** Walk the list following next pointers, incrementing a counter for each node.

```

typedef struct s_list
{
    struct s_list *next;
    void          *data;
} t_list;

int ft_list_size(t_list *begin_list)
{
    int n = 0;
    while (begin_list)           // walk until end of list
    {
        n++;
        begin_list = begin_list->next;
    }
    return (n);
}

```

ft_range

- **File:** ft_range.c | **Allowed:** malloc

Subject

Write the following function:

```
int *ft_range(int start, int end);
```

It must allocate (with malloc()) an array of integers, fill it with consecutive values that begin at start and end at end (Including start and end !), then return a pointer to the first value of the array.

Examples:

- With (1, 3) you will return an array containing 1, 2 and 3.
- With (-1, 2) you will return an array containing -1, 0, 1 and 2.
- With (0, 0) you will return an array containing 0.
- With (0, -3) you will return an array containing 0, -1, -2 and -3.

- **Approach:** Compute size as $|end - start| + 1$. Fill the array stepping +1 or -1 depending on direction.

```

#include <stdlib.h>
int *ft_range(int start, int end)
{
    int size = (start <= end ? end - start : start - end) + 1; // number of elements
    int *arr = malloc(size * sizeof(int));
    int i = 0;
    while (i < size)
    {
        arr[i] = start;
        start += (start <= end) ? 1 : -1; // step toward end
        i++;
    }
    return (arr);
}

```

ft_rrange

- **File:** ft_rrange.c | **Allowed:** malloc

Subject

Write the following function:

```
int    *ft_rrange(int start, int end);
```

It must allocate (with malloc()) an array of integers, fill it with consecutive values that begin at end and end at start (Including start and end !), then return a pointer to the first value of the array.

Examples:

- With (1, 3) you will return an array containing 3, 2 and 1.
- With (-1, 2) you will return an array containing 2, 1, 0 and -1.
- With (0, 0) you will return an array containing 0.
- With (0, -3) you will return an array containing -3, -2, -1 and 0.

- **Approach:** Same size calculation as ft_range, but fill starting from end and stepping toward start.

```
#include <stdlib.h>
int *ft_rrange(int start, int end)
{
    int size = (start <= end ? end - start : start - end) + 1; // number of elements
    int *arr = malloc(size * sizeof(int));
    int step = (start <= end) ? -1 : 1; // direction: end → start
    int i = 0;
    while (i < size)
    {
        arr[i] = end; // fill from end side
        end += step;
        i++;
    }
    return (arr);
}
```

hidenp

- **File:** hidenp.c | **Allowed:** write

Subject

Write a program named hidenp that takes two strings and displays 1 followed by a newline if the first string is hidden in the second one, otherwise displays 0 followed by a newline.

Let s1 and s2 be strings. We say that s1 is hidden in s2 if it's possible to find each character from s1 in s2, in the same order as they appear in s1. Also, the empty string is hidden in any string.

If the number of parameters is not 2, the program displays a newline.

Examples:

```
$>./hidenp "fgex.;" "tyf34gdgf;ektufjhgdgex.;.rtjynur6" | cat -e
```

```
1$
$>./hiddenp "abc" "2altrb53c.sse" | cat -e
1$
$>./hiddenp "abc" "btarc" | cat -e
0$
$>./hiddenp | cat -e
$
```

- **Approach:** Two-pointer scan. Walk s2, advance the s1 pointer whenever there's a match. After the loop, if s1 is exhausted all its characters were found in order.

```
#include <unistd.h>
int main(int ac, char **av)
{
    if (ac == 3)
    {
        char *s1 = av[1];
        char *s2 = av[2];
        while (*s1 && *s2)
            if (*s1 == *s2++) s1++;           // advance s1 on match, always advance s2
        write(1, (*s1 == '\0') ? "1" : "0", 1); // s1 exhausted → all chars found
    }
    write(1, "\n", 1);
}
```

lcm

- **File:** lcm.c | **Allowed:** none

Subject

Write a function who takes two unsigned int as parameters and returns the computed LCM of those parameters.

LCM (Lowest Common Multiple) of two non-zero integers is the smallest positive integer divisible by both integers.

If at least one integer is null, LCM is equal to 0.

Your function must be prototyped as follows:

```
unsigned int    lcm(unsigned int a, unsigned int b);
```

- **Approach:** Start from the larger of the two values and increment until a number divisible by both is found.

```
unsigned int lcm(unsigned int a, unsigned int b)
{
    unsigned int n;
    if (a == 0 || b == 0) return (0);           // zero input → lcm is 0
    n = (a > b) ? a : b;                         // start from the larger value
    while (n % a != 0 || n % b != 0) n++;       // increment until divisible by both
    return (n);
}
```

paramsum

- **File:** paramsum.c | **Allowed:** write

Subject

Write a program that displays the number of arguments passed to it, followed by a newline.

If there are no arguments, just display a 0 followed by a newline.

Example:

```
$> ./paramsum 1 2 3 5 7 24
6
$> ./paramsum 6 12 24 | cat -e
3$
$> ./paramsum | cat -e
0$
```

- **Approach:** `argc - 1` equals the number of arguments (excluding the program name). Print it.

```
#include <unistd.h>
void putnbr(int n)
{
    if (n >= 10) putnbr(n / 10);           // print leading digits recursively
    char c = n % 10 + '0';
    write(1, &c, 1);
}
int main(int ac, char **av)
{
    (void)av;
    putnbr(ac - 1);                       // argc - 1 = number of arguments
    write(1, "\n", 1);
}
```

pgcd

- **File:** `pgcd.c` | **Allowed:** `printf`, `atoi`, `malloc`, `free`

Subject

Write a program that takes two strings representing two strictly positive integers that fit in an `int`.

Display their highest common denominator followed by a newline.

If the number of parameters is not 2, display a newline.

Examples:

```
$> ./pgcd 42 10 | cat -e
2$
$> ./pgcd 42 12 | cat -e
6$
$> ./pgcd 14 77 | cat -e
7$
$> ./pgcd 17 3 | cat -e
1$
$> ./pgcd | cat -e
$
```

- **Approach:** Euclid's algorithm — repeatedly replace (a, b) with $(b, a \% b)$ until $b == 0$. The result is a .

```

#include <stdio.h>
#include <stdlib.h>
int main(int ac, char **av)
{
    if (ac == 3)
    {
        int a = atoi(av[1]);
        int b = atoi(av[2]);
        while (b) { int t = b; b = a % b; a = t; } // Euclid's algorithm
        printf("%d", a);                          // a is the GCD
    }
    printf("\n");
}

```

print_hex

- **File:** print_hex.c | **Allowed:** write

Subject

Write a program that takes a positive (or zero) number expressed in base 10, and displays it in base 16 (lowercase letters) followed by a newline.

If the number of parameters is not 1, the program displays a newline.

Examples:

```

$> ./print_hex "10" | cat -e
a$
$> ./print_hex "255" | cat -e
ff$
$> ./print_hex "5156454" | cat -e
4eae66$
$> ./print_hex | cat -e
$

```

- **Approach:** Recursive function — print $n / 16$ first (for leading digits), then print the hex character for $n \% 16$.

```

#include <unistd.h>
void print_hex(unsigned long n)
{
    char *hex = "0123456789abcdef";
    if (n >= 16) print_hex(n / 16);      // print higher digits first
    write(1, &hex[n % 16], 1);         // print current digit
}
int ft_atoi(char *s)
{
    int n = 0;
    while (*s >= '0' && *s <= '9') n = n * 10 + (*s++ - '0');
    return (n);
}
int main(int ac, char **av)
{
    if (ac == 2) print_hex(ft_atoi(av[1]));
    write(1, "\n", 1);
}

```

rstr_capitalizer

- **File:** rstr_capitalizer.c | **Allowed:** write

Subject

Write a program that takes one or more strings and, for each argument, puts the last character that is a letter of each word in uppercase and the rest in lowercase, then displays the result followed by a \n.

A word is a section of string delimited by spaces/tabs or the start/end of the string.

If there are no parameters, display \n.

Examples:

```
$> ./rstr_capitalizer | cat -e
```

```
$
```

```
$> ./rstr_capitalizer "a FiRst LiTTle TEST" | cat -e
```

```
A firSt littLe tesT$
```

```
$> ./rstr_capitalizer "SecONd teST A LiTTle BiT  Moar comPLEX" "  But... This iS not THAT COMPLEX"
```

```
e
```

```
seconD tesT A littLe biT  moaR compleX$
```

```
but... thiS iS noT thaT compleX$
```

- **Approach:** Lowercase every letter. Then check if the next character ends the word (space, tab, or end of string) — if so, uppercase the current letter.

```
#include <unistd.h>
void rstr_cap(char *str)
{
    int i = 0;
    while (str[i])
    {
        if (str[i] == ' ' || str[i] == '\t') { write(1, &str[i], 1); i++; continue; }
        while (str[i] && str[i] != ' ' && str[i] != '\t')
        {
            char c = str[i];
            if (c >= 'A' && c <= 'Z') c += 32; // lowercase all letters
            // check if this is the last letter of the word
            int is_last = (!str[i + 1] || str[i + 1] == ' ' || str[i + 1] == '\t');
            if (is_last && c >= 'a' && c <= 'z') c -= 32; // uppercase last letter
            write(1, &c, 1);
            i++;
        }
    }
    write(1, "\n", 1);
}

int main(int ac, char **av)
{
    if (ac == 1) write(1, "\n", 1);
    else { int i = 1; while (i < ac) rstr_cap(av[i++]); }
}
```

str_capitalizer

- **File:** str_capitalizer.c | **Allowed:** write

Subject

Write a program that takes one or several strings and, for each argument, capitalizes the first character of each word (if it's a letter), puts the rest in lowercase, and displays the result on the standard output, followed by a `\n`.

A "word" is defined as a part of a string delimited either by spaces/tabs, or by the start/end of the string.

If there are no arguments, the program must display `\n`.

Example:

```
$> ./str_capitalizer | cat -e
```

```
$
```

```
$> ./str_capitalizer "a FiRSt LiTTle TESt" | cat -e
```

```
A First Little Test$
```

```
$> ./str_capitalizer "__SecONd teST A LiTTle BiT  Moar comPLEX" "  But... This iS not THAT COMPLE
```

```
e
```

```
__second Test A Little Bit  Moar Complex$
```

```
But... This Is Not That Complex$
```

- **Approach:** Track if we're at the start of a new word. Uppercase the first letter of each word, lowercase the rest.

```
#include <unistd.h>
void str_cap(char *str)
{
    int i = 0;
    int new_word = 1;           // start of string counts as a new word
    while (str[i])
    {
        char c = str[i];
        if (c == ' ' || c == '\t') { new_word = 1; write(1, &c, 1); i++; continue; }
        if (new_word && c >= 'a' && c <= 'z') c -= 32;           // uppercase first letter
        else if (!new_word && c >= 'A' && c <= 'Z') c += 32;    // lowercase the rest
        new_word = 0;
        write(1, &c, 1);
        i++;
    }
    write(1, "\n", 1);
}
int main(int ac, char **av)
{
    if (ac == 1) write(1, "\n", 1);
    else { int i = 1; while (i < ac) str_cap(av[i++]); }
}
```

tab_mult

- **File:** tab_mult.c | **Allowed:** write

Subject

Write a program that displays a number's multiplication table.

The parameter will always be a strictly positive number that fits in an int, and said number times 9 will also fit in an int.

If there are no parameters, the program displays `\n`.

Examples:

```
$>./tab_mult 9
1 x 9 = 9
2 x 9 = 18
[...]
9 x 9 = 81
$>./tab_mult | cat -e
$
```

- **Approach:** Loop i from 1 to 9. For each iteration print $i \times n = i*n$ using `putnbr` (no `printf` allowed).

```
#include <unistd.h>
void putnbr(int n)
{
    if (n >= 10) putnbr(n / 10);           // print leading digits recursively
    char c = n % 10 + '0';
    write(1, &c, 1);
}
int ft_atoi(char *s)
{
    int n = 0;
    while (*s >= '0' && *s <= '9') n = n * 10 + (*s++ - '0');
    return (n);
}
int main(int ac, char **av)
{
    if (ac == 2)
    {
        int n = ft_atoi(av[1]);
        int i = 1;
        while (i <= 9)
        {
            putnbr(i); write(1, " x ", 3); putnbr(n);
            write(1, " = ", 3); putnbr(i * n); write(1, "\n", 1);
            i++;
        }
    }
    else write(1, "\n", 1);
}
```

Level 3

flood_fill

- **File:** `flood_fill.c` | **Allowed:** none

Subject

Write a function that takes a `char**` as a 2-dimensional array of `char`, a `t_point` as the dimensions of this array and a `t_point` as the starting point.

Starting from the given 'begin' `t_point`, this function fills an entire zone by replacing characters inside with the character 'F'. A zone is a group of the same character delimited horizontally and vertically by other characters or the array boundary.

The flood_fill function won't fill diagonally.

```
void flood_fill(char **tab, t_point size, t_point begin);
```

```
typedef struct s_point
{
    int x;
    int y;
} t_point;
```

Example:

```
t_point size = {8, 5};
```

```
char *zone[] = {
```

```
    "11111111",
```

```
    "10001001",
```

```
    "10010001",
```

```
    "10110001",
```

```
    "11100001",
```

```
};
```

```
t_point begin = {7, 4};
```

```
// After flood_fill:
```

```
FFFFFFFF
```

```
F000F00F
```

```
F00F000F
```

```
F0FF000F
```

```
FFF0000F
```

- **Approach:** Recursive DFS. Save the target character at begin, then recursively fill all 4 neighbours that share the same character.

```
typedef struct s_point { int x; int y; } t_point;
```

```
void fill(char **tab, t_point size, t_point cur, char target)
```

```
{
```

```
    // stop if out of bounds or not the target character
```

```
    if (cur.y < 0 || cur.y >= size.y || cur.x < 0 || cur.x >= size.x)
```

```
        return;
```

```
    if (tab[cur.y][cur.x] != target)
```

```
        return;
```

```
    tab[cur.y][cur.x] = 'F'; // mark as filled
```

```
    fill(tab, size, (t_point){cur.x - 1, cur.y}, target); // left
```

```
    fill(tab, size, (t_point){cur.x + 1, cur.y}, target); // right
```

```
    fill(tab, size, (t_point){cur.x, cur.y - 1}, target); // up
```

```
    fill(tab, size, (t_point){cur.x, cur.y + 1}, target); // down
```

```
}
```

```
void flood_fill(char **tab, t_point size, t_point begin)
```

```
{
```

```
    fill(tab, size, begin, tab[begin.y][begin.x]); // start fill from begin point
```

```
}
```

fprime

- **File:** fprime.c | **Allowed:** printf, atoi

Subject

Write a program that takes a positive int and displays its prime factors on the

standard output, followed by a newline.

Factors must be displayed in ascending order and separated by '*', so that the expression in the output gives the right result.

If the number of parameters is not 1, simply display a newline.

Examples:

```
$> ./fprime 225225 | cat -e
3*3*5*5*7*11*13$
$> ./fprime 42 | cat -e
2*3*7$
$> ./fprime 1 | cat -e
1$
$> ./fprime | cat -e
$
$> ./fprime 42 21 | cat -e
$
```

- **Approach:** Start with factor 2. If $n \% f == 0$, print it and divide n by f (don't increment — same factor may repeat). Otherwise try the next factor.

```
#include <stdio.h>
#include <stdlib.h>
int main(int ac, char **av)
{
    if (ac == 2)
    {
        int n = atoi(av[1]);
        int f = 2;
        int first = 1; // tracks whether to print '*' before factor
        if (n == 1) printf("1"); // special case: 1 has no prime factors
        while (f <= n)
        {
            if (n % f == 0) // f divides n: it's a factor
            {
                if (!first) printf("*");
                printf("%d", f);
                first = 0;
                n /= f; // divide out the factor, try same f again
            }
            else f++; // f doesn't divide: try next candidate
        }
        printf("\n");
    }
}
```

ft_itoa

- **File:** ft_itoa.c | **Allowed:** malloc

Subject

Write a function that takes an int and converts it to a null-terminated string. The function returns the result in a char array that you must allocate.

Your function must be declared as follows:

```
char    *ft_itoa(int nbr);
```

- **Approach:** Count the number of digits (including sign). Allocate the buffer. Fill from the end using % 10 and / 10. Handle 0 and negatives specially.

```
#include <stdlib.h>
char *ft_itoa(int n)
{
    int len = (n <= 0) ? 1 : 0;           // reserve space for '-' or '0'
    int tmp = n;
    while (tmp) { len++; tmp /= 10; }     // count digits
    char *s = malloc(len + 1);
    s[len] = '\0';
    if (n == 0) { s[0] = '0'; return (s); }
    if (n < 0) s[0] = '-';                // write minus sign
    while (n)
    {
        s[--len] = (n < 0 ? -(n % 10) : n % 10) + '0'; // fill from right
        n /= 10;
    }
    return (s);
}
```

ft_list_foreach

- **Files:** ft_list_foreach.c, ft_list.h | **Allowed:** none

Subject

Write a function that takes a list and a function pointer, and applies this function to each element of the list.

```
void    ft_list_foreach(t_list *begin_list, void (*f)(void *));
```

The function pointed to by f will be used as follows:

```
(*f)(list_ptr->data);
```

You must use the following structure, and turn it in as ft_list.h:

```
typedef struct    s_list
{
    struct s_list *next;
    void          *data;
} t_list;
```

- **Approach:** Walk the list and call f(node->data) for each node.
- **Note:** Submit both ft_list_foreach.c **and** ft_list.h.

ft_list.h:

```
typedef struct s_list
{
    struct s_list *next;
    void          *data;
} t_list;
```

ft_list_foreach.c:

```
#include "ft_list.h"
void ft_list_foreach(t_list *begin_list, void (*f)(void *))
```

```

{
    while (begin_list)
    {
        f(begin_list->data);           // apply function to current node's data
        begin_list = begin_list->next; // advance to next node
    }
}

```

ft_list_remove_if

- **File:** ft_list_remove_if.c | **Allowed:** free

Subject

Write a function called ft_list_remove_if that removes from the passed list any element the data of which is "equal" to the reference data.

```
void ft_list_remove_if(t_list **begin_list, void *data_ref, int (*cmp)());
```

cmp takes two void* and returns 0 when both parameters are equal.

You have to use the ft_list.h file, which will contain:

```

typedef struct      s_list
{
    struct s_list   *next;
    void             *data;
    t_list;
}

```

- **Approach:** Recursive. If the current node matches, unlink it, free it, and recurse on the updated head. Otherwise recurse on next.
- **Note:** ft_list.h is provided by the exam — do **not** submit it.

```

#include <stdlib.h>
#include "ft_list.h"

void ft_list_remove_if(t_list **begin_list, void *data_ref, int (*cmp)(void *, void
↪ *))
{
    if (!begin_list || !*begin_list) return; // empty list: nothing to do
    t_list *cur = *begin_list;
    if (cmp(cur->data, data_ref) == 0) // current node matches: remove it
    {
        *begin_list = cur->next; // unlink node
        free(cur);              // free memory
        ft_list_remove_if(begin_list, data_ref, cmp); // check new head
    }
    else
        ft_list_remove_if(&cur->next, data_ref, cmp); // check rest of list
}

```

ft_split

- **File:** ft_split.c | **Allowed:** malloc

Subject

Write a function that takes a string, splits it into words, and returns them as

a NULL-terminated array of strings.

A "word" is defined as a part of a string delimited either by spaces/tabs/new lines, or by the start/end of the string.

Your function must be declared as follows:

```
char    **ft_split(char *str);
```

- **Approach:** Two passes — first count the words to allocate the array, then fill it by duplicating each word.

```
#include <stdlib.h>
static int is_sep(char c) { return (c == ' ' || c == '\t' || c == '\n'); }
static int wordlen(char *s) { int n = 0; while (s[n] && !is_sep(s[n])) n++; return n; }
static int count_words(char *s)
{
    int n = 0;
    // skip separators, count each word
    while (*s) { while (*s && is_sep(*s)) s++; if (*s) { n++; while (*s &&
        ↪ !is_sep(*s)) s++; } }
    return n;
}
char **ft_split(char *str)
{
    int nw = count_words(str);
    char **arr = malloc((nw + 1) * sizeof(char *)); // +1 for NULL terminator
    arr[nw] = NULL;
    int wi = 0;
    while (*str)
    {
        while (*str && is_sep(*str)) str++;           // skip separators
        if (!*str) break;
        int len = wordlen(str);
        arr[wi] = malloc(len + 1);                   // allocate word + '\0'
        arr[wi][len] = '\0';
        int j = 0;
        while (j < len) { arr[wi][j] = str[j]; j++; } // copy word
        str += len;
        wi++;
    }
    return (arr);
}
```

rev_wstr

- **File:** rev_wstr.c | **Allowed:** write, malloc, free

Subject

Write a program that takes a string as a parameter, and prints its words in reverse order.

A "word" is a part of the string bounded by spaces and/or tabs, or the begin/end of the string.

If the number of parameters is different from 1, the program will display '\n'.

In the parameters that are going to be tested, there won't be any "additional" spaces (no spaces at beginning/end, words always separated by exactly one space).

Examples:

```
$> ./rev_wstr "You hate people! But I love gatherings. Isn't it ironic?" | cat -e
ironic? it Isn't gatherings. love I But people! hate You$
$> ./rev_wstr "abcdefghijklm"
abcdefghijklm
$> ./rev_wstr "Wingardium Leviosa" | cat -e
Leviosa Wingardium$
$> ./rev_wstr | cat -e
$
```

- **Approach:** Walk backwards from the end of the string. Each time you find a word boundary, print from start to end and add a space separator if needed.

```
#include <unistd.h>
int ft_strlen(char *s) { int i = 0; while (s[i]) i++; return i; }
int main(int ac, char **av)
{
    if (ac == 2)
    {
        char *str = av[1];
        int len = ft_strlen(str);
        int i = len - 1;
        while (i >= 0)
        {
            int end = i;
            while (i >= 0 && str[i] != ' ') i--; // find word start
            int start = i + 1;
            write(1, &str[start], end - start + 1); // print word
            if (i > 0) write(1, " ", 1);           // space between words
            i--;
        }
        write(1, "\n", 1);
    }
}
```

rostring

- **File:** rostring.c | **Allowed:** write, malloc, free

Subject

Write a program that takes a string and displays this string after rotating it one word to the left.

Thus, the first word becomes the last, and others stay in the same order.

A "word" is defined as a part of a string delimited either by spaces/tabs, or by the start/end of the string.

Words will be separated by only one space in the output.

If there's less than one argument, the program displays \n.

Example:

```
$>./rostring "abc  " | cat -e
abc$
$>./rostring "Que la      lumiere soit et la lumiere fut"
la lumiere soit et la lumiere fut Que
$>./rostring "      AkjhZ zLKIJz , 23y"
zLKIJz , 23y AkjhZ
$>./rostring | cat -e
$
```

- **Approach:** Record where the first word starts and ends. Then print words 2...n in order (epur_str style), and append the first word at the end.

```
#include <unistd.h>
int main(int ac, char **av)
{
    int i = 0;
    if (ac >= 2)
    {
        char *s = av[1];
        while (s[i] == ' ' || s[i] == '\t') i++; // skip leading whitespace
        int first_start = i;
        while (s[i] && s[i] != ' ' && s[i] != '\t') i++; // find end of first word
        int first_end = i;
        int first = 1; // tracks separator printing
        int j = first_end;
        while (s[j]) // print remaining words
        {
            while (s[j] == ' ' || s[j] == '\t') j++;
            if (!s[j]) break;
            if (!first) write(1, " ", 1);
            first = 0;
            while (s[j] && s[j] != ' ' && s[j] != '\t')
                write(1, &s[j++], 1);
        }
        if (first_end > first_start) // append first word at end
        {
            if (!first) write(1, " ", 1);
            write(1, s + first_start, first_end - first_start);
        }
    }
    write(1, "\n", 1);
}
```

sort_int_tab

- **File:** sort_int_tab.c | **Allowed:** none

Subject

Write the following function:

```
void sort_int_tab(int *tab, unsigned int size);
```

It must sort (in-place) the 'tab' int array, that contains exactly 'size' members, in ascending order.

Doubles must be preserved.
Input is always coherent.

- **Approach:** Bubble sort — compare adjacent elements, swap if out of order, restart from index 0. Repeat until a full pass has no swaps.

```
void sort_int_tab(int *tab, unsigned int size)
{
    unsigned int i = 0;
    while (i + 1 < size)
    {
        if (tab[i] > tab[i + 1])           // swap if out of order
        {
            int tmp = tab[i];
            tab[i] = tab[i + 1];
            tab[i + 1] = tmp;
            i = 0;                          // restart from beginning after swap
        }
        else i++;
    }
}
```

sort_list

- **File:** sort_list.c | **Allowed:** none

Subject

Write the following function:

```
t_list *sort_list(t_list* lst, int (*cmp)(int, int));
```

This function must sort the list given as a parameter, using the function pointer `cmp` to select the order to apply, and returns a pointer to the first element of the sorted list.

Duplications must remain.

You must use the type `t_list` described in the file `list.h` that is provided to you (`#include "list.h"`). Do not turn it in.

Functions passed as `cmp` will return a value different from 0 if `a` and `b` are in the right order, 0 otherwise.

```
int ascending(int a, int b) { return (a <= b); }
```

`list.h` content:

```
typedef struct s_list t_list;
struct s_list { int data; t_list *next; };
```

- **Approach:** Bubble sort on the list — swap data values (not nodes) when `cmp` returns 0. Restart from head after each swap.
- **Note:** `list.h` is provided by the exam — do **not** submit it.

```
#include "list.h"
t_list *sort_list(t_list *lst, int (*cmp)(int, int))
{
    t_list *head = lst;           // keep reference to head
    while (lst && lst->next)
    {
        if (!cmp(lst->data, lst->next->data)) // wrong order: swap data
        {
            int tmp = lst->data;
            lst->data = lst->next->data;
            lst->next->data = tmp;
        }
        lst = lst->next;
    }
    return head;
}
```

```

    {
        int tmp = lst->data;
        lst->data = lst->next->data;
        lst->next->data = tmp;
        lst = head;                // restart from beginning
    }
    else lst = lst->next;          // correct order: advance
}
return (head);
}

```

Quick Tips

- Always end output with `\n` (even if no valid input).
- `argc` counts the program name: `argc == 2` means 1 argument.
- For bit operations: `>>` shifts right, `<<` shifts left, `&` masks, `|` ORs.
- `'a' + 'z' - c` mirrors a lowercase letter; `'A' + 'Z' - c` mirrors uppercase.
- GCD via Euclid: `while (b) { t = b; b = a % b; a = t; } → a is GCD.`
- Bubble sort restarts from index 0 after each swap.
- `ft_list_foreach` requires submitting **both** `ft_list_foreach.c` and `ft_list.h`.
- `ft_list_remove_if` and `sort_list` use a provided header — do **not** submit it.